- 학습 데이터를 더 넣으면 일반적으로 예측의 불확실성이 줄어들고 정답에 가까워질 수 있음!
- 단, 조건이 있음.
- → 학습 데이터가 실제 예측 대상과 같은 분포에서 왔고, 그 분포를 대표할 수 있어야 함.
- → 모델의 표현 능력과 학습 방법도 충분히 뒷받침되어야 함.
- 이런 조건이 만족되면 데이터가 늘어날수록 일반화 성능이 좋아지고 예측의 불확실성이 줄어드는 경향이 있음.
- 하지만 데이터만 무조건 늘린다고 성능 향상이 보장되는 것은 아님!
- → 데이터의 대표성, 모델의 귀납적 편향, 학습 과정도 중요함.
- 적은 데이터 + 귀납적 가설이 맞는 모델
- → 적은 데이터로도 더 좋은 예측이 가능하다
- ≡ 데이터를 더 넣거나 or 더 좋은 아키텍처를 만들던가
- EX
- SIREN(Sinusoidal Representation Networks)
- → 사인 함수(sin)를 활성화 함수로 사용하는 신경망!
- → 암시적 신경 표현(Implicit Neural Representation)의 한 종류임.
- → 좌표를 입력받아 해당 좌표의 값을 출력하는 연속 함수를 학습함. (좌표 → 값)
- EX
- → 이미지: (x, y) 좌표 → 해당 위치의 RGB 값
- → 오디오: 시간 t → 해당 시점의 신호 값
- → 3D 형상: (x, y, z) 좌표 → 해당 위치의 형상 관련 값
- 즉, 이미지 전용 모델이 아니라 이미지·오디오·3D 형상 등 다양한 연속 신호를 함수로 fitting하는 모델!

---

- CNN
- 이미지를 인식하려면?
  1. 이미지 자체를 패치로 나눠 → classifier을 돌려 각 패치가 어떤 것인지 확인한다
- 화질을 높일려면? 패치를 더 나누면 됨. 근데 그러면 식별이 어려워 짐
  1. Convolution
- 우리가 아는 cnn에서의 filter를 쓰면 됨
- 합성곱(Convolution)도 행렬 곱으로 표현할 수 있음!
- 입력을 펼치면 Toeplitz 구조의 가중치 행렬로 나타낼 수 있음. (2D·다채널에서는 block-Toeplitz 구조)
- 하지만 일반적인 Fully Connected Layer와 같지는 않음!
- → 지역성(Locality): 각 출력이 입력의 일부 영역에만 연결됨. (희소 연결)
- → 가중치 공유(Weight Sharing): 같은 필터를 여러 위치에 반복 적용함.
- 즉, 합성곱은 지역성과 가중치 공유라는 제약을 가진 Fully Connected Layer의 특수한 형태(부분집합)라고 볼 수 있음.
- → Convolution은 지역 연결(Local Connectivity) + 가중치 공유(Weight Sharing)를 사용하는 구조!

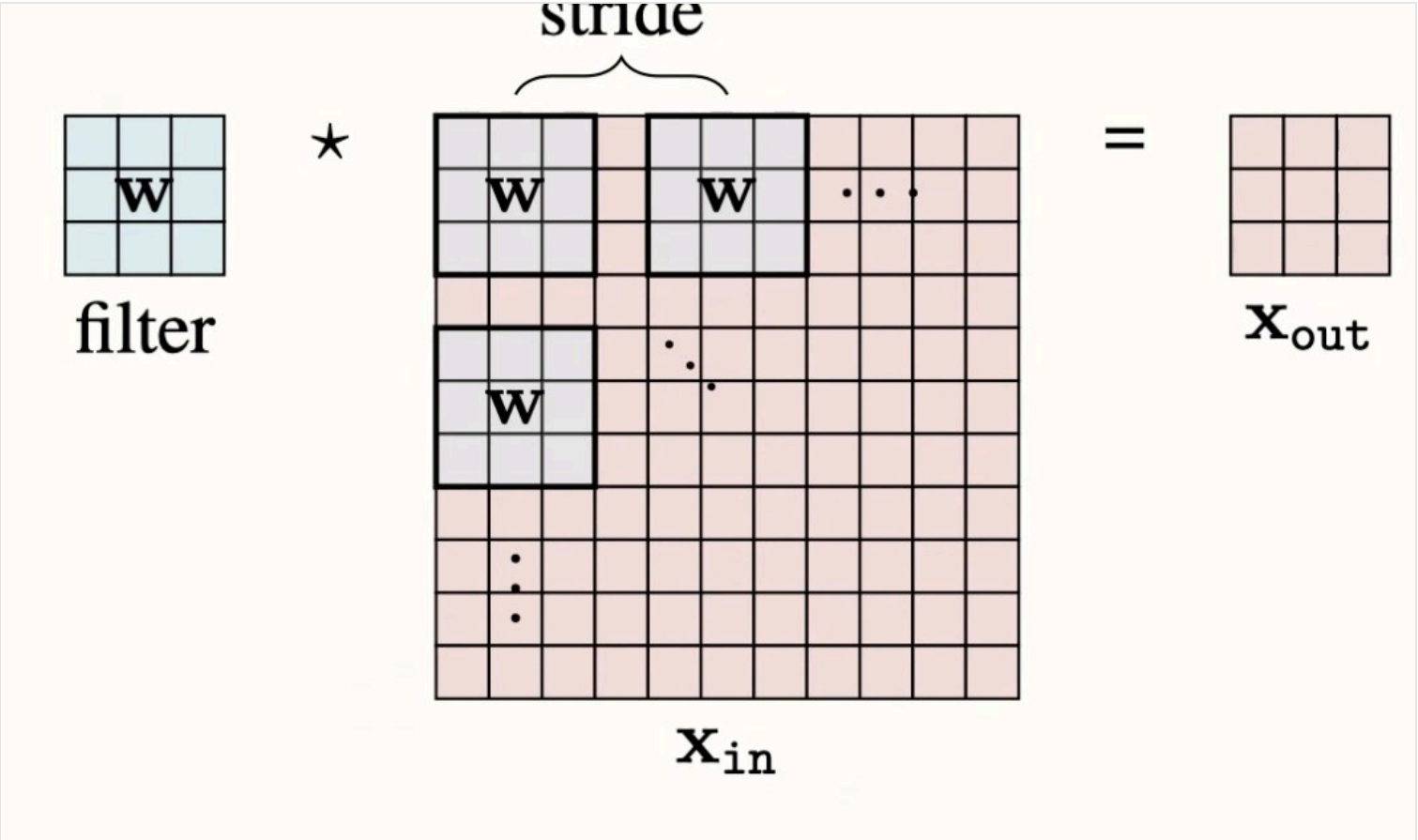
- Locally Connected Layer도 주변의 일부 영역에만 연결되지만, 위치마다 서로 다른 가중치를 사용함. (가중치 공유 X)
- 반면 Convolution Layer는 같은 필터의 가중치를 여러 위치에서 공유함.
- 즉, 둘 다 지역 연결을 사용하지만 가중치 공유 여부가 핵심 차이!

---

- color?
- input이 색이 있음 :color → conv → 1 dim
  output이 색이 필요함 1dim → conv → color
- general
  → multi-input, mult-output

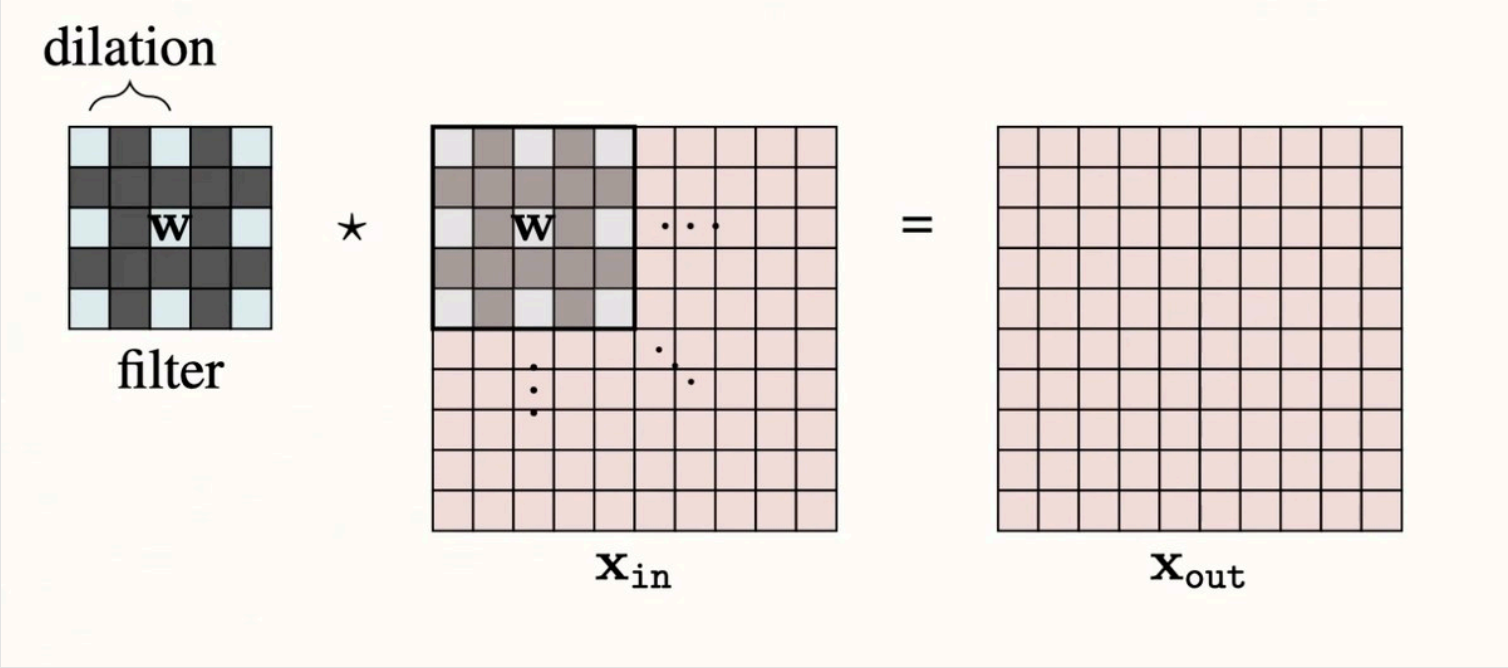
---

- pooling
- 1) max pooling
  2) mean pooling
- 출력의 안정성 강화
- 채널들을 통합할 수도 있음
- 다운 샘플링(차원 낮춤) 도 할 수 있음
- 스트라이드 연산
- conv + 풀링 + 다운샘플링




- Stride(스트라이드)
- → Convolution에서 필터를 한 번 이동할 때 몇 칸씩 움직이는지를 나타냄.

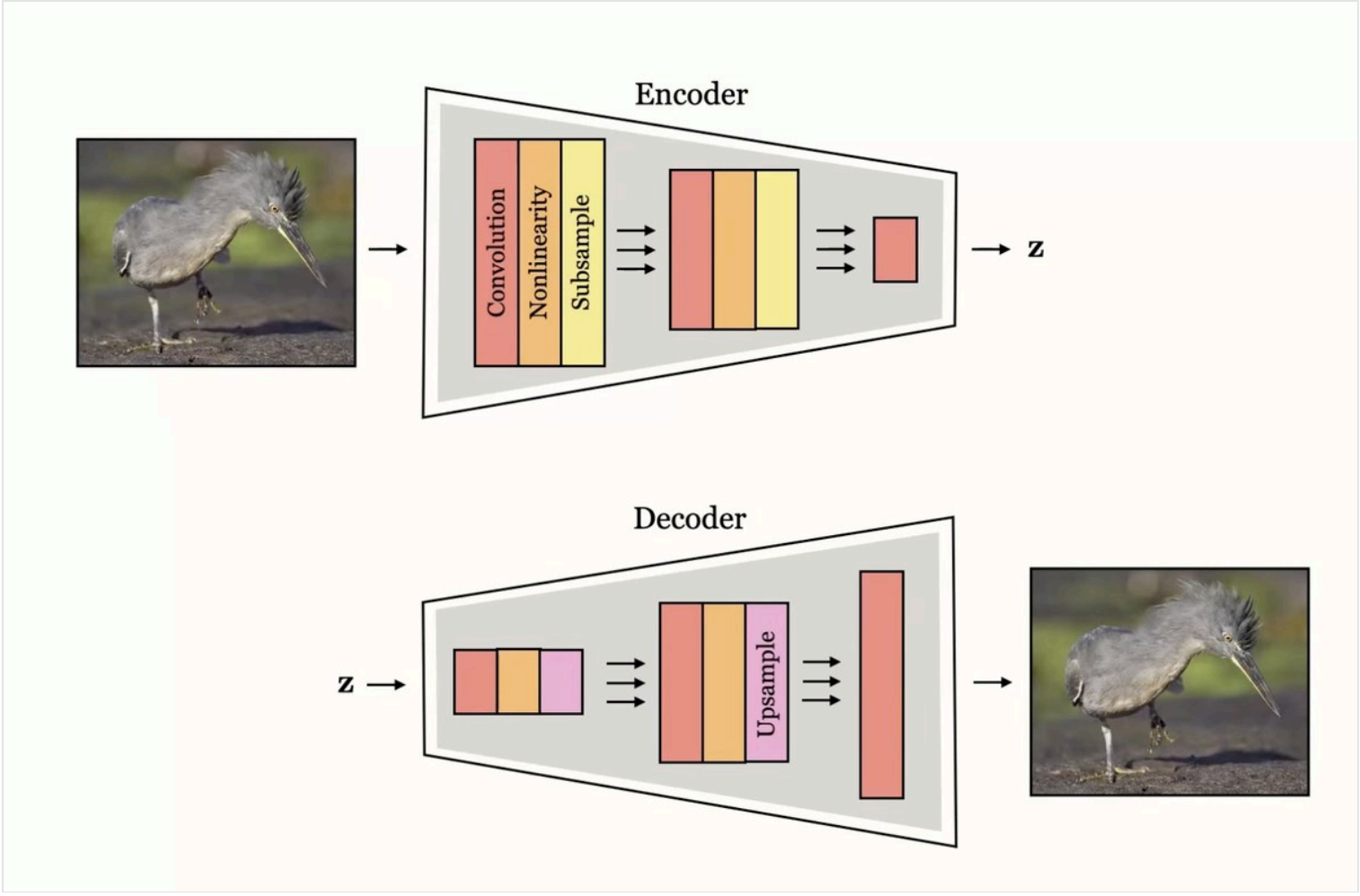
- → Stride = 1이면 필터를 한 칸씩 이동함.
- → Stride가 커지면 필터가 더 큰 간격으로 이동하므로 출력의 공간 크기가 작아짐. (다운샘플링 효과)
- 바로 앞 그림의 경우
- → 입력 크기: 11×11
- → 필터 크기: 3×3
- → Stride = 4
- → Padding = 0 (없음)
- → 출력 크기: 3×3
- 출력 크기 공식 (한 축 기준)
- Output Size = ⌊(Input Size − Filter Size + 2×Padding) / Stride⌋ + 1
- 그림에 적용하면
- ⌊(11−3)/4⌋+1 = 3
- → 가로·세로 각각 3이므로 출력은 3×3!
- 즉, 이 그림에서는 필터를 가로·세로 방향으로 4칸씩 이동하여 11×11 입력을 3×3 출력으로 줄인 것임.
- Stride를 크게 하면 출력 크기와 연산량을 줄일 수 있지만, 일부 위치의 정보를 놓칠 수도 있음.
- Dilated filter



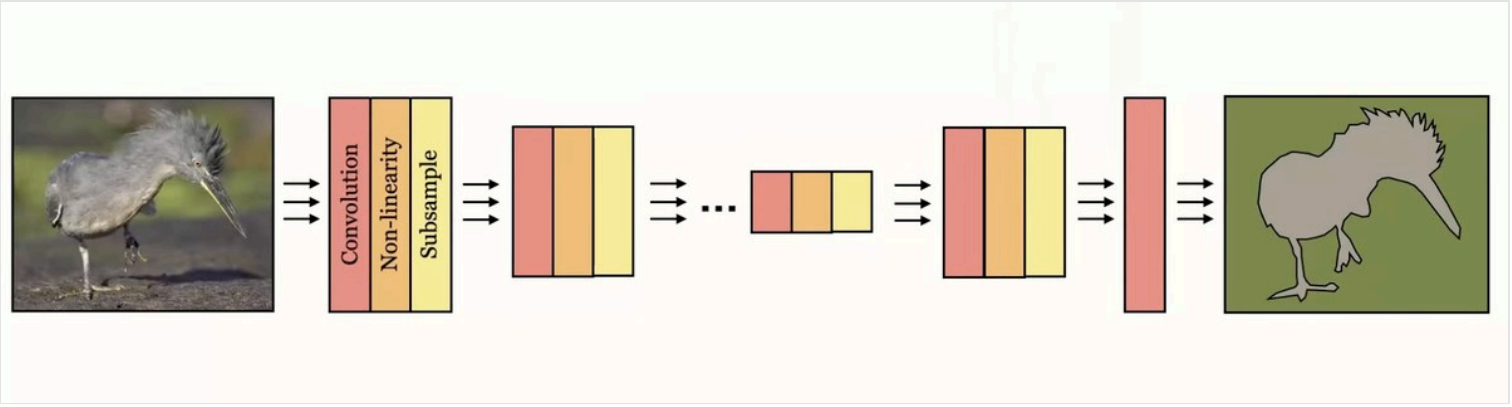

- 몇개를 0으로 한 큰 필터
  컨텍스트 창이 늘어난다

---

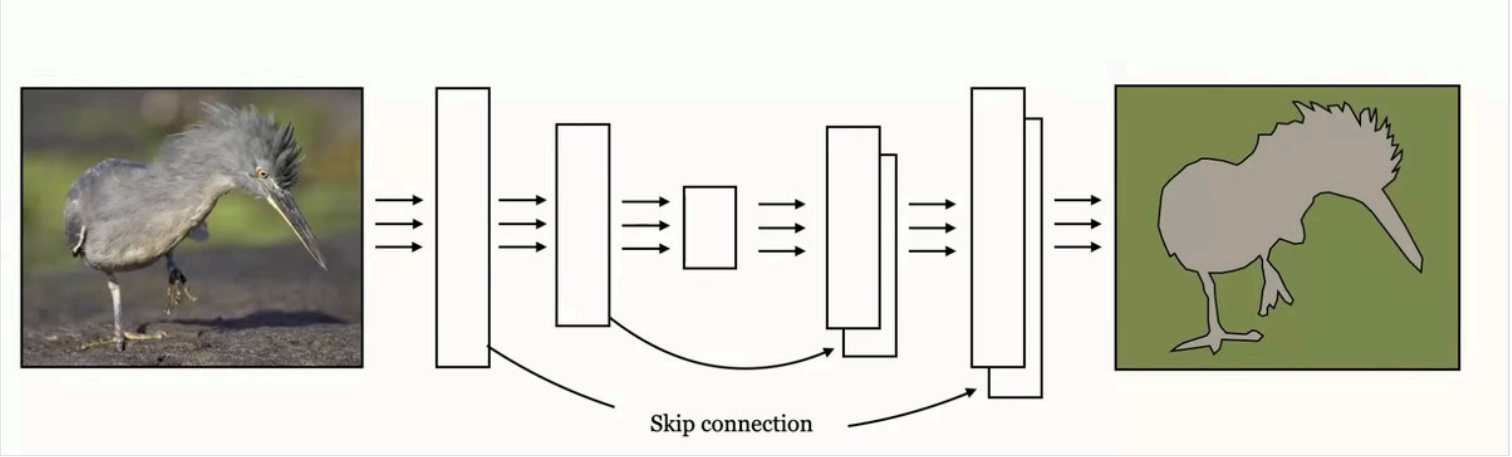
- popular CNN Architectures
- Encoder-Decoder Architectures



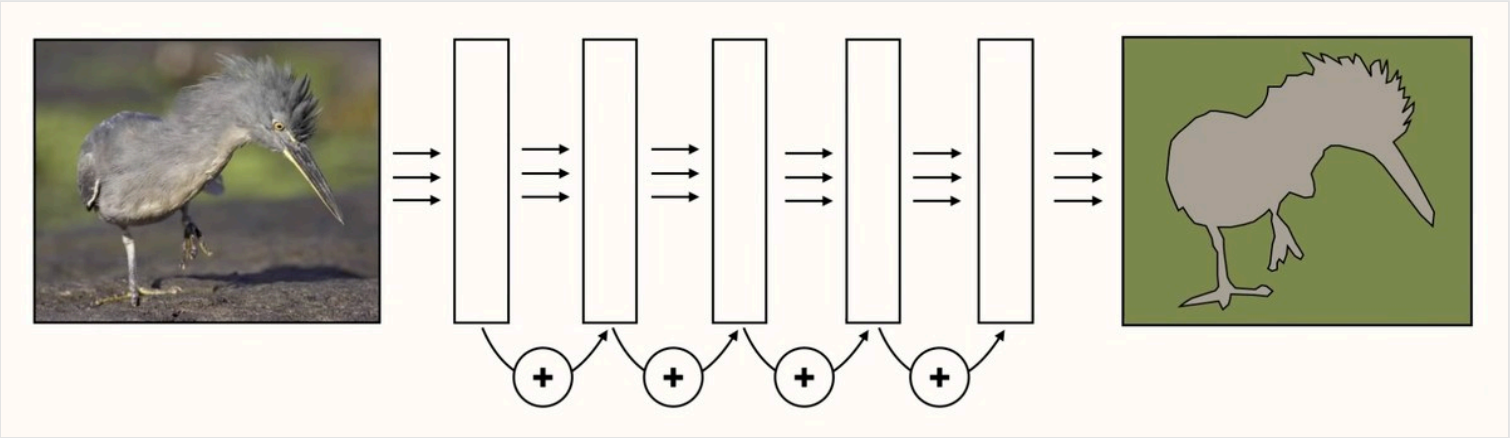

- Encoder: 원래 이미지 → 낮은 차원의 Z (다운샘플링)
  Decoder: Z → 이미지 예측 (업샘플링)
- Decoder의 출력이 반드시 Encoder의 입력과 일치해야 하는 것은 아님!
- Autoencoder의 경우
- → Encoder: 입력 x → 잠재 표현 z
- → Decoder: z → 복원된 입력 $\hat{x}$
- → 목표: $\hat{x}$가 원래 입력 x와 최대한 비슷해지도록 학습함. (입력 재구성)
- 하지만 일반적인 Encoder-Decoder 구조에서는 입력과 다른 출력을 만드는 것도 가능함!
- EX
- → Segmentation: 입력 이미지 → 픽셀별 클래스 예측
- → Image-to-Image Translation: 입력 이미지 → 변환된 이미지
- 즉, 입력 재구성은 Autoencoder의 목표이고, Encoder-Decoder 전체에 공통인 목표는 아님!
- Decoder는 이미지를 만들어 낼 수도 있음
- Image-to-image




- segmentation도 가능!
- (before)차원을 줄이지 않음.(고해상도 유지 → 비용이 커짐)
- after → U-net




- Skip connection이라는 연결이 있음.
  중간 layer를 건너 뛰는 형태
- 저차원 - 특징 정보 / 고차원 - 세부 정보 모두 얻을 수 있는 장점이 있음
- ResNet



- ResNet은 잔차 블록(Residual Block) 단위로 Skip Connection을 사용함.
- 입력 x를 두 경로로 전달함.
- → Residual Path: F(x)를 학습함.
- → Skip Path: 입력 x를 그대로 전달함. (Identity Mapping)
- 두 경로의 출력을 더함!
- y = F(x) + x (입출력 차원이 같은 경우)

- 차원이 다르면 Skip Path에 Projection 등을 적용해 차원을 맞출 수 있음.
- 이때 처리할지/스킵할지 선택하는 게 아님!
- → 두 경로는 항상 합산됨. (별도의 게이팅·선택 구조가 아님)
- 장점
- → Skip Path를 통해 기울기가 더 잘 전달되어 깊은 네트워크의 학습이 쉬워짐.
- → F(x)=0에 가까워지면 항등함수 y≈x를 쉽게 표현할 수 있음.
- 즉, 최적의 깊이를 자동으로 선택하는 구조가 아니라 잔차 학습으로 깊은 네트워크의 최적화를 돕는 구조!

---

- 차원 추가
- Convolution은 기본적으로 평행이동 등변성(Translation Equivariance)을 가짐.
- → 입력 이미지가 이동하면 출력 특징 맵도 그에 따라 이동함. (stride=1, 경계 효과 등을 제외한 경우)
- Translation Invariance(평행이동 불변성)와는 다름!
- → Equivariance: 입력이 이동하면 출력도 대응해서 이동함.
- → Invariance: 입력이 이동해도 출력이 같게 유지됨.
- Pooling 등을 사용하면 작은 위치 변화에 대해 근사적인 불변성을 얻을 수 있음. (항상 보장되는 것은 아님)
- 주의할 점!
- → Stride나 Pooling으로 다운샘플링할 때 Aliasing이 발생할 수 있음.
- → 이 때문에 작은 위치 이동에도 출력이 달라져 등변성이나 기대한 불변성이 깨질 수 있음.
- 위치 정보를 명시적으로 사용하고 싶다면?
- 1. MLP 사용
- → 일반적인 MLP는 평행이동 불변성이나 등변성을 구조적으로 강제하지 않음.
- → 대신 CNN의 지역성·가중치 공유라는 귀납적 편향도 포기하는 선택임.
- 2. Positional Encoding 사용
- → 입력 특징에 위치 정보를 추가하여 모델이 위치를 구분할 수 있게 함.
- → 위치 인코딩은 학습 가능한 형태(Learned)도 있고, 고정된 형태(Fixed)도 있음.
- → Sinusoidal Positional Encoding은 학습하지 않는 고정형의 대표적인 예!
- 즉, Positional Encoding은 위치를 반드시 추가로 학습하는 방식이 아니라, 위치 정보를 모델에 제공하는 장치임.
- Neural Fields